

实验 1 ROS 通信机制

一、实验目的

- 1、熟练掌握 ROS 常用的通信机制：话题通信和服务通信。
- 2、编写代码实现两种通信机制对应的案例。
- 3、熟练使用 ROS 中的一些常用操作命令，如 `roslaunch`、`rostopic`、`roscat` 等。

二、实验环境与实验器材

- 1、实验硬件设备
计算机
- 2、实验软件配置
Ubuntu20.04, ros-noetic

三、实验内容

1、控制小乌龟做圆周运动，并获取乌龟位姿

- 1) 启动小乌龟

终端 1 输入以下命令：

<code>\$ roscore</code>	//启动 ROS 核心服务
-------------------------	---------------

终端 2 输入以下命令：

<code>\$ roslaunch turtlesim turtlesim_node</code>	//启动小乌龟节点
--	-----------

- 2) 通过 ROS 的操作命令，查看不同节点的各类消息

终端 3 输入以下命令：

<code>\$ roscat</code>	//查看活动的节点
<code>\$ rostopic list (或 rostopic list -v)</code>	//查看活跃的话题(详情)
<code>\$ rostopic info /turtle1/cmd_vel</code>	//查看话题相关消息的类型、发布方和订阅方信息
<code>\$ rostopic type /turtle1/cmd_vel</code>	
<code>\$ rostopic info geometry_msgs/Twist</code>	//查看消息格式
<code>\$ rqt_graph</code>	//计算图：可视化节点与话题之间的通信关系

- 3) 向小乌龟的订阅方发布一条运动消息，控制小乌龟运动

以 10Hz 的频率循环发送运动信息，终端 4 输入以下命令：

<code>\$ rostopic pub -r 10 /turtle1/cmd_vel geometry_msgs/Twist</code> "linear: x:1.0 y:0.0 z:0.0 angular: x:0.0 y:0.0 z:0.5"
--

- 4) 通过 ROS 命令，获取乌龟发布的位姿信息，以及不同节点各类消息的变化
终端 5 输入以下命令：

```
$ rostopic type /turtle1/pose //查看话题相关消息的类型
$ rosmmsg info turtlesim/Pose //查看消息格式
$ rostopic echo /turtle1/pose //实时查看话题发布的消息
$ rqt_graph //计算图：可视化节点与话题之间的通信关系
```

- 5) 关闭 ros 系统

通过快捷方式 `ctrl+c`，先逐个关闭启动的节点，再关闭 `roscore`。

2、实现基本的话题通信，一方发布数据，一方接收数据

话题通信是 ROS 中使用频率最高的一种通信模式，是基于发布订阅的数据交互模式，即一个节点发布消息，另一个节点订阅该消息。

要求：编写发布订阅实现，发布方以 10Hz 的频率发送文本消息，订阅方接收文本消息并将消息内容打印输出。

具体流程如下：

第 1 步：创建 ros 功能包。

- 1) 创建工作空间并初始化

命令如下：

```
$ mkdir -p ~/catkin_plumbing/src
$ cd ~/catkin_plumbing
$ catkin_make //初始化工作空间
```

- 2) 创建 ros 功能包

进入 `src` 文件夹，创建功能包 `plumbing_pub_sub`，并添加依赖 `roscpp` 和 `std_msgs`。

命令如下：

```
$ cd src
$ catkin_create_pkg plumbing_pub_sub roscpp std_msgs
```

第 2 步：编辑源文件。

进入功能包 `plumbing_pub_sub` 的 `src` 目录编辑源文件，命令如下：

```
$ cd plumbing_pub_sub/src
$ gedit demo01_pub.cpp //用 gedit 编辑发布方源文件
$ gedit demo02_sub.cpp //用 gedit 编辑订阅方源文件
```

注：如系统中未安装 `gedit` 文本编辑器，可用 `vim` 或 `nano`，或安装 `gedit`。

安装命令如下：

```
$ sudo apt install gedit
```

`demo01_pub.cpp` 文件输入内容示例以下：

```
// 1.包含头文件
#include "ros/ros.h"
#include "std_msgs/String.h" //普通文本类型的消息
#include <sstream>
```

```

int main(int argc, char *argv[])
{
    //设置编码
    setlocale(LC_ALL, "");

    //2.初始化 ROS 节点:命名(唯一)
    // 参数 1 和参数 2 后期为节点传值会使用
    // 参数 3 是节点名称, 是一个标识符, 需要保证运行后, 在 ROS 网络
    拓扑中唯一
    ros::init(argc, argv, "talker");
    //3.实例化 ROS 句柄
    ros::NodeHandle nh; //该类封装了 ROS 中的一些常用功能

    //4.实例化 发布者 对象
    //泛型: 发布的消息类型
    //参数 1: 要发布到的话题
    //参数 2: 队列中最大保存的消息数, 超出此阈值时, 先进的先销毁(时
    间早的先销毁)
    ros::Publisher pub = nh.advertise<std_msgs::String>("chatter", 10);

    //5.组织被发布的数据, 并编写逻辑发布数据
    //数据(动态组织)
    std_msgs::String msg;
    // msg.data = "你好啊!!! ";
    std::string msg_front = "Hello 你好! "; //消息前缀
    int count = 0; //消息计数器

    //逻辑(一秒 10 次)
    ros::Rate r(1);

    //节点不死
    while (ros::ok())
    {
        //使用 stringstream 拼接字符串与编号
        std::stringstream ss;
        ss << msg_front << count;
        msg.data = ss.str();
        //发布消息
        pub.publish(msg);
        //加入调试, 打印发送的消息
        ROS_INFO("发送的消息:%s", msg.data.c_str());

        //根据前面制定的发送频率自动休眠 休眠时间 = 1/频率;
    }
}

```

```

        r.sleep();
        count++;//循环结束前，让 count 自增
        //暂无应用
        ros::spinOnce();
    }

    return 0;
}

```

demo02_sub.cpp 文件输入内容示例如下：

```

// 1.包含头文件
#include "ros/ros.h"
#include "std_msgs/String.h"

void doMsg(const std_msgs::String::ConstPtr& msg_p){
    ROS_INFO("我听见:%s",msg_p->data.c_str());
    // ROS_INFO("我听见:%s",(*msg_p).data.c_str());
}

int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    //2.初始化 ROS 节点:命名(唯一)
    ros::init(argc,argv,"listener");
    //3.实例化 ROS 句柄
    ros::NodeHandle nh;

    //4.实例化 订阅者 对象
    ros::Subscriber sub = nh.subscribe<std_msgs::String>("chatter",10,doMsg);
    //5.处理订阅的消息(回调函数)

    //6.设置循环调用回调函数
    ros::spin();//循环读取接收的数据，并调用回调函数处理

    return 0;
}

```

第 3 步：配置 CMakeList.txt。

进入功能包 plumbing_pub_sub，修改 CMakeList.txt，命令如下：

```

$ cd ~/catkin_plumbing/src/plumbing_pub_sub
$ gedit CMakeList.txt

```

修改内容如下：

```
//声明 C++可执行文件
//找到 add_executable 所在位置（约第 136 行），添加
add_executable(demo01_pub_node src/demo01_pub.cpp)
add_executable(demo02_sub_node src/demo02_sub.cpp)

//指定库、可执行文件的链接库
//找到 target_link_libraries 所在位置（约第 150 行），添加
target_link_libraries(demo01_pub_node
    ${catkin_LIBRARIES}
)
target_link_libraries(demo02_sub_node
    ${catkin_LIBRARIES}
)
```

第 4 步：编译并执行。

1) 编译

```
$ cd ~/catkin_plumbing
$ catkin_make                    //编译整个工作空间
或 $ catkin_make --pkg plumbing_pub_sub    //只编译某个特定功能包
```

2) 命令行执行

终端 1 输入以下命令：

```
$ roscore                        //启动 ROS 核心服务
```

终端 2 输入以下命令：

```
$ cd ~/catkin_plumbing
$ source ./devel/setup.bash      //设置 ROS 所需的环境变量
$ rosrn plumbing_pub_sub demo01_pub_node    //启动 demo01_pub_node 节点
```

终端 3 输入以下命令：

```
$ cd ~/catkin_plumbing
$ source ./devel/setup.bash      //设置 ROS 所需的环境变量
$ rosrn plumbing_pub_sub demo02_sub_node    //启动 demo02_sub_node 节点
```

3) 编写 launch 文件并执行

```
$ cd ~/catkin_plumbing/src/plumbing_pub_sub
$ mkdir launch
$ cd launch
$ gedit start_pub_sub.launch
```

start_pub_sub.launch 文件输入内容示例以下：

```
<launch>
  <node pkg="plumbing_pub_sub" type="demo01_pub_node" name="pub_node" output="screen" />
  <node pkg="plumbing_pub_sub" type="demo02_sub_node" name="sub_node" output="screen" />
</launch>
```

执行，终端输入以下命令：

```
$ cd ~/catkin_plumbing
$ source ./devel/setup.bash
```

```
$ roslaunch plumbing_pub_sub start_pub_sub.launch
```

3、实现基本的服务通信，Client 发送请求，Server 接收请求并产生响应

服务通信也是 ROS 中一种极其常用的通信模式，服务通信基于请求响应模式，是一种应答机制。即一个节点 A 向另一个节点 B 发送请求，B 接收处理请求并产生响应结果返回给 A。

要求：编写服务通信实现，客户端提交两个整数至服务端，服务端求和并响应结果到客户端。

具体流程如下：

第 1 步：创建 ros 功能包。

1) 创建工作空间并初始化

```
$ mkdir -p ~/catkin_plumbing2/src  
$ cd ~/catkin_plumbing2  
$ catkin_make
```

注：如果不想创建新的工作空间，也可在已有的工作空间下（如：catkin_plumbing）创建 ros 功能包。

2) 创建 ros 功能包

进入 src 文件夹，创建功能包 plumbing_server_client，并添加依赖 roscpp、rospy 和 std_msgs。命令如下

```
$ cd src  
$ catkin_create_pkg plumbing_server_client roscpp rospy std_msgs
```

第 2 步：，创建自定义服务消息

服务消息为服务器与客户端通信的数据载体，包含服务和请求两部分。

1) 定义服务消息 srv 文件

进入功能包新建 srv 目录，按固定格式添加 srv 文件 AddInts.srv

```
$ cd ~/catkin_plumbing2/src/plumbing_server_client  
$ mkdir srv  
$ cd srv  
$ gedit AddInts.srv
```

AddInts.srv 文件输入内容示例以下：

```
# 客户端请求时发送的两个数字  
int32 num1  
int32 num2  
---  
# 服务器响应发送的数据  
int32 sum
```

注：请求部分和响应部分用三个“-”（连字符）分割。

2) 编辑配置文件 package.xml 和 CMakeList.txt

```
$ cd ~/catkin_plumbing2/src/plumbing_server_client
```

```
$ gedit package.xml
$ gedit CMakeList.txt
```

package.xml 文件修改内容如下：

```
//添加编译依赖
//找到 build_depend 所在位置（约第 51 行），添加
<build_depend>message_generation</build_depend>

//添加执行依赖
//找到 exec_depend 所在位置（约第 59 行），添加
<exec_depend>message_runtime</exec_depend>
```

CMakeList.txt 文件修改内容如下：

```
//添加构建依赖 message_generation
//找到 find_package 所在位置（约第 10 行），添加 message_generation
find_package(catkin REQUIRED COMPONENTS
    roscpp
    rospy
    std_msgs
    message_generation
) //需要加入 message_generation，必须有 std_msgs

//找到 add_service_files 所在位置（约第 58 行），添加
add_service_files(
    FILES
    AddInts.srv
)

//找到 generate_messages 所在位置（约第 76 行），添加
generate_messages(
    DEPENDENCIES
    std_msgs
)

//添加运行时的依赖（这部分可不修改）
//找到 catkin_package 所在位置（约第 114 行），修改
catkin_package (
    CATKIN_DEPENDS roscpp rospy std_msgs message_runtime
)
```

3) 编译

```
$ cd ~/catkin_plumbing2
```

```
$ catkin_make
```

查看编译后的中间文件，所在路径：~/catkin_plumbing2/devel/include/plumbing_server_client，该路径下的 AddInts.h、AddIntsRequest.h、AddIntsResponse.h 头文件是后续 C++ 运行中需要调用的中间文件。

第 3 步：编辑源文件

进入功能包 plumbing_server_client 的 src 目录编辑源文件，命令如下：

```
$ cd plumbing_server_client/src
$ gedit demo01_server.cpp          //用 gedit 编辑服务端源文件
$ gedit demo02_client.cpp          //用 gedit 编辑客户端源文件
```

demo01_server.cpp 文件输入内容示例以下：

```
// 1.包含头文件
#include "ros/ros.h"
#include "plumbing_server_client/AddInts.h"

// bool 返回值由于标志是否处理成功
bool doReq(plumbing_server_client::AddInts::Request& req,
           plumbing_server_client::AddInts::Response& resp){
    int num1 = req.num1;
    int num2 = req.num2;

    ROS_INFO("服务器接收到的请求数据为:num1 = %d, num2 = %d",num1,
num2);

    //逻辑处理
    if (num1 < 0 || num2 < 0)
    {
        ROS_ERROR("提交的数据异常:数据不可以为负数");
        return false;
    }

    //如果没有异常，那么相加并将结果赋值给 resp
    resp.sum = num1 + num2;
    return true;
}

int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    // 2.初始化 ROS 节点
    ros::init(argc,argv,"AddInts_Server");
    // 3.创建 ROS 句柄
    ros::NodeHandle nh;
```



```

// 4.创建 服务 对象
ros::ServiceServer server = nh.advertiseService("AddInts",doReq);
ROS_INFO("服务已经启动....");
//      5.回调函数处理请求并产生响应
//      6.由于请求有多个，需要调用 ros::spin()
ros::spin();
return 0;
}

```

demo02_client.cpp 文件输入内容示例以下：

```

// 1.包含头文件
#include "ros/ros.h"
#include " plumbing_server_client/AddInts.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");

    // 调用时动态传值
    if (argc != 3)
    {
        ROS_ERROR("请提交两个整数");
        return 1;
    }

    // 2.初始化 ROS 节点
    ros::init(argc,argv,"AddInts_Client");
    // 3.创建 ROS 句柄
    ros::NodeHandle nh;
    // 4.创建 客户端 对象
    ros::ServiceClient client = nh.serviceClient<plumbing_server_client::AddInts>("AddInts");

    // 5.组织请求数据
    demo03_server_client::AddInts ai;
    ai.request.num1 = atoi(argv[1]);
    ai.request.num2 = atoi(argv[2]);
    // 6.发送请求,返回 bool 值，标记是否成功
    bool flag = client.call(ai);
    // 7.处理响应
    if (flag)
    {
        ROS_INFO("请求正常处理,响应结果:%d",ai.response.sum);
    }
    else
    {

```

```

        ROS_ERROR("请求处理失败....");
        return 1;
    }

    return 0;
}

```

第 4 步：配置 CMakeList.txt。

进入功能包 `plumbing_server_client`，修改 `CMakeList.txt`，命令如下：

```

$ cd ~/catkin_plumbing2/src/plumbing_server_client
$ gedit CMakeList.txt

```

修改内容如下：

```

//声明 C++可执行文件
//找到 add_executable 所在位置（约第 145 行），添加
add_executable(demo01_server_node src/demo01_server.cpp)
add_executable(demo02_client_node src/demo02_client.cpp)

//添加可执行文件的 CMake 目标依赖
//找到 add_dependencies 所在位置（约第 157 行），添加
add_dependencies(demo01_server_node ${PROJECT_NAME}_gencpp)
add_dependencies(demo02_client_node ${PROJECT_NAME}_gencpp)

//指定库、可执行文件的链接库
//找到 target_link_libraries 所在位置（约第 162 行），添加
target_link_libraries(demo01_server_node
    ${catkin_LIBRARIES}
)
target_link_libraries(demo02_client_node
    ${catkin_LIBRARIES}
)

```

第 5 步：编译并执行。

1) 编译

```

$ cd ~/catkin_plumbing2
$ catkin_make                                //编译整个工作空间
或 $ catkin_make --pkg plumbing_server_client //只编译某个特定功能包

```

2) 命令行执行

终端 1 输入以下命令：

```

$ roscore                                //启动 ros 核心服务

```

终端 2 输入以下命令：

```

$ cd ~/catkin_plumbing2
$ source ./devel/setup.bash
$ rosrn plumbing_server_client demo01_server_node

```

终端 3 输入以下命令：

```

$ cd ~/catkin_plumbing2

```

```
$ source ./devel/setup.bash
$ rosrn plumbing_server_client demo02_client_node 参数 1 参数 2
```

四、实验要求

1、配置 ros 环境

给电脑配置 ros-noetic 实验环境。

2、仿真实验

- 1) 控制小乌龟做圆周运动，并获取乌龟位姿。
- 2) 编写发布订阅实现，发布方以 10Hz 的频率发送文本消息，订阅方接收文本消息并将消息内容打印输出。
- 3) 编写服务通信实现，客户端提交两个整数至服务端，服务端求和并响应结果到客户端。
- 4) 编写仿真实验 1) 和 3) 的 launch 文件。
- 5) 用 python 语言实现仿真实验 2) 和 3)。

3、思考问题

- 1) ros::init()函数、launch 文件和在 CMakeList.txt 指定可执行文件的链接库即添加 target_link_libraries()时，三处都对节点进行命名，这三个节点命名的作用是什么？有什么区别和联系
- 2) 一个 ros 功能包可以注册多少节点？
- 3) 用 roslaunch 命令启动节点时，需不需要手动运行 roscore？为什么？
- 4) 比较话题通信中的自定义消息和服务通信中的自定义消息。
- 5) 自定义消息在配置 CMakeList.txt 时，为什么要添加 add_dependencies()？它的作用是什么？
- 6) 服务通信中，如先启动客户端，再启动服务端，是否运行成功？如果失败，给出原因和处理方式。

五、实验报告要求

1. 提交时间：两周之内提交
2. 页数：4~6 页以内
3. 报告内容：实验目的、实验步骤与结果分析、结论与问题讨论
4. 提交方式：电子版和纸质版均需提交；

电子版：实验报告和代码打包，加密压缩后，上传至网盘 <http://58.206.101.71:8266/>；

压缩包名：班级+学号+姓名，报告格式为 word 或 pdf；

纸质版：附上电子版密码，黑白双面打印即可，由各班学委收齐后交至科学馆 303。

六、参考资料

1. 张新钰，赵虚左，丘楠，郭世纯，《ROS 机器人理论与实践》，清华大学出版社

2. 课程资料:

<http://www.autolabor.com.cn/book/ROSTutorials/>

3. 课程讲解 (B 站):

https://www.bilibili.com/video/BV1Ci4y1L7ZZ/?spm_id_from=333.999.0.0&vd_source=4913ab2f45996351f46d15faddba2bfd

注: 本次实验主要涉及《ROS 机器人理论与实践》第一章至第四章内容, 重点参阅第二章第 1、2 和 4 节内容。